

DYNAMIC PROGRAMMING

13th September 2026

SO, WHERE AM I WRITING MY CODE?

Dynamic programming (DP) is a general methodology for solving sequential decision making problems. The framework is applicable whenever a problem can be decomposed into a sequence of stages, with a decision required at each stage that influences future outcomes. Dynamic programming has found applications in a wide range of areas including optimal control, quantitative finance, supply chain management, *etc.* Although the underlying principles are conceptually straightforward, practical implementation often requires careful mathematical modelling and the logical reasoning can be rigorous.

It is worth noting that, prior to writing this article, the closest mathematical topic I had studied was linear programming. Without reading Bertsekas' *Dynamic Programming and Optimal Control*, I would not have been able to develop an understanding of the subject. I found learning about dynamic programming thoroughly enjoyable. The topic is remarkably broad, and this article has only touched upon its most fundamental ideas. There remain so many important concepts, applications, *etc.* that could not be explored within what I have written.

DYNAMIC PROGRAMMING

A discrete-time dynamic programming model consists of two fundamental components:

1. The underlying discrete time dynamic system.
2. An objective function that is additive over time.

A dynamic programming problem can be divided into stages with a decision required at each stage. The decision at one stage transforms the current state into a state in the next stage. Consider a finite horizon problem with $N + 1$ stages (or time periods) labelled $0, 1, \dots, N - 1, N$. At each stage k , the system occupies a state x_k , and a decision (or control), u_k , is selected from an admissible set of controls, $U_k(x_k)$. The evolution of the system is also affected by a random parameter, w_k , also known as a disturbance. The state transition can be expressed as $x_{k+1} = f(x_k, u_k, w_k)$ for any stage $k = 0, \dots, N - 1$. Throughout, we assume that the system satisfies the Markov property, meaning that all information relevant for future evolution is contained in the current state, x_k . Consequently, the next state depends only on the current state, the current decision, and the current disturbance, rather than the entire history of the process. That is, the state of next stage, x_{k+1} , is determined as a function of the current state x_k , current decision u_k (the choice we make at stage k from the available options) and the random variable w_k (the probability distribution of w_k often depends on x_k and u_k).

The objective is to minimise a cumulative cost, or equivalently, maximise a cumulative profit. Let $g_k(x_k, u_k, w_k)$ denote the cost incurred at stage k , and let $g_N(x_N)$ denote the terminal cost at the final stage. For a policy (or strategy), $\pi = \{u_0, u_1, \dots, u_{N-1}\}$, the total cost is $g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k, w_k)$. The optimal policy, π^* , is obtained by solving $J_{\pi^*}(x_0) = \min_{\pi} \left\{ \mathbb{E} \left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k, w_k) \right] \right\}$, where the expectation is taken with respect to the random disturbances. The function, $J_{\pi}(x_0)$, represents the expected total cost associated with policy, π , when the initial state is x_0 .

2.1 Dynamic programming algorithm

The key idea underlying dynamic programming is Bellman's **Principle of Optimality**, which states that an optimal policy possesses the property that, regardless of the initial state and decisions, the remaining decisions must themselves constitute an optimal policy for the subproblem beginning at the resulting state. That is, if $\pi^* = \{u_0^*, u_1^*, \dots, u_{N-1}^*\}$ is optimal for the original problem, then for any stage i , the tail policy $\pi_i^* = \{u_i^*, u_{i+1}^*, \dots, u_{N-1}^*\}$ must be optimal for the optimisation problem beginning at stage i .

This principle allows a large optimisation problem to be decomposed into a sequence of smaller and more manageable subproblems. To exploit this property, dynamic programming introduces the value function (also known as the cost-to-go function), $J_k(x_k)$, defined as the minimum expected future cost incurred from stage k onwards, given that the system is currently in state x_k . At the terminal stage, there are no remaining decisions to make, so the value function is simply $J_N(x_N) = g_N(x_N)$. The value functions at earlier stages are obtained recursively using the Bellman equation, $J_k(x_k) = \min_{u_k \in U_k(x_k)} \{ \mathbb{E} [g_k(x_k, u_k, w_k) + J_{k+1}(f(x_k, u_k, w_k))] \}$, $k = N - 1, \dots, 0$. This recursion expresses the fact that the optimal cost at stage k consists of the immediate cost incurred at that stage together with the optimal future cost from stage $k + 1$ onwards. By solving the

Bellman equation backwards from stage N to stage 0, you can obtain both the optimal value function and the corresponding optimal policy.

At the final stage, the optimisation problem is simplest because there are no future decisions to consider. Once the optimal cost has been determined for every possible terminal state, the optimal decisions at the preceding stage can be computed. Repeating this backward induction process stage by stage eventually yields the optimal decision at the initial state. Thus, dynamic programming can transform a high-dimensional optimisation problem into a sequence of simpler optimisation problems, making otherwise intractable problems computationally manageable.

AN EXAMPLE

Consider an investor with an initial wealth of £1m. The investment horizon consists of two periods. At the beginning of each period, the investor may allocate all wealth to either a risk-free asset returning 2% per period, or a risky asset returning either 10% or -5% with equal probability. What is the maximum expected terminal wealth by the end of the second period?

Solution: Let x_k denote the investor's wealth at time k , and let the decision/control variable, u_k , represent the investment choice, that is, $u_k = \begin{cases} 0, & \text{risk-free asset} \\ 1, & \text{non risk-free asset} \end{cases}$.

The state transition equation is $x_{k+1} = \begin{cases} 1.02x_k, & u_k = 0 \\ 1.1x_k, & u_k = 1 \text{ with probability } \frac{1}{2} \\ 0.95x_k, & u_k = 1 \text{ with probability } \frac{1}{2} \end{cases}$.

Since the objective is to maximise terminal wealth at the end of the second period, the terminal value function is $J_2(x_2) = x_2$. Using Bellman's principle of optimality, we work backwards from the final period. At time $k = 1$, $J_1(x_1) = \max_{u_1} \mathbb{E}[J_2(x_2)]$. If the investor chooses the risk-free asset, $\mathbb{E}[x_2] = 1.02x_1$. If the investor chooses the non risk-free asset, $\mathbb{E}[x_2] = (\frac{1}{2})(1.1x_1) + (\frac{1}{2})(0.95x_1) = 1.025x_1$. Thus, since $1.025x_1 > 1.02x_1$, the non risk-free asset is optimal. Therefore, $J_1(x_1) = 1.025x_1$. Now to the initial period, the Bellman equation gives $J_0(x_0) = \max_{u_0} \mathbb{E}[J_1(x_1)]$. If the investor chooses the risk-free asset, $J_0(x_0) = (1.025)(1.02x_0) = 1.0455x_0$. If the investor chooses the non risk-free asset, $J_0(x_0) = (\frac{1}{2})(1.025)(1.1x_0) + (\frac{1}{2})(1.025)(0.95x_0) = 1.050625x_0$. Since, $1.050625x_0 > 1.0455x_0$, the non risk-free asset is the optimal choice. For initial wealth, $x_0 = £1m$, the optimal expected terminal wealth is $J_0(£1m) = £1.050625m$, increasing wealth by £50,625. That is, the optimal policy is to invest in the non risk-free asset during both periods. This backward approach, also referred to as the tail policy, yields the answer and strategy for investing.

CONCLUSION

Dynamic programming is incredibly useful when solving sequential decision making problems by decomposing a complex optimisation problem into a collection of simpler subproblems. Through Bellman's principle of optimality and the associated value function, optimal decisions can be determined efficiently using backward induction. It is worth noting that dynamic programming provides the foundation for much of modern optimal control theory. In continuous time settings, the same principle leads to the Hamilton-Jacobi-Bellman (HJB) equation.

REFERENCES

- Bertsekas, Dimitri P. (2017). *Dynamic Programming and Optimal Control, Volume I*. 4th ed. Belmont, MA: Athena Scientific. ISBN: 978-1-886529-43-4.
- Zhou, Xinfeng (2008). *A Practical Guide to Quantitative Finance Interviews*. CreateSpace Independent Publishing Platform. ISBN: 9781438236667.

ALEXANDER FLANN